

Comparatif des Architectures Web : Client-Serveur, Microservices et DApps

Introduction Générale

Les **architectures web** définissent la manière dont les composants d'une application (données, logique métier, interface utilisateur) interagissent entre eux. Trois modèles dominent aujourd'hui le développement web :

1. Architecture Client-Serveur

C'est le modèle traditionnel du web. L'application est divisée en deux parties : le **client** (navigateur ou application front-end) qui envoie des requêtes, et le **serveur** (back-end) qui répond avec des données.

Exemple : un site e-commerce classique comme Amazon.

2. Architecture Microservices

Elle découpe une application en plusieurs **services indépendants**, chacun ayant une fonction précise (authentification, paiement, gestion produit...). Ces services communiquent via des API.

Exemple : Netflix, Amazon ou Uber utilisent cette architecture.

3. Architecture DApp (Application Décentralisée)

Elle repose sur la **blockchain**. Les données et la logique ne sont pas hébergées sur un serveur central, mais distribuées entre plusieurs nœuds du réseau.

Exemple : Uniswap, OpenSea ou toute application sur Ethereum.

Différences Principales

Aspect	Client-Serveur	Microservices	DApps
Organisation	Centralisée	Semi-décentralisée	Totalement décentralisée
Gestion des données	Serveur unique	Plusieurs services autonomes	Réseau distribué (Blockchain)
Scalabilité	Limitée	Élevée	Dépend de la blockchain
Sécurité	Gérée côté serveur	Par service/API	Cryptographie et consensus

Aspect	Client-Serveur	Microservices	DApps
Maintenance	Simple mais rigide	Flexible mais complexe	Complexe (smart contracts im- muables)

1. Architecture Client-Serveur

Avantages

- Simplicité de conception et de déploiement
- Contrôle total des données et du serveur
- Facilité de maintenance et de mise à jour centralisée
- Idéale pour les petites et moyennes applications

Inconvénients

- Point unique de défaillance (si le serveur tombe, tout s'arrête)
- Scalabilité limitée
- Dépendance forte au serveur central

Technologies Utilisées

- **Front-end (client)** : HTML, CSS, JavaScript, React, Vue.js, Angular
- **Back-end (serveur)** : Node.js, PHP, Python (Django, Flask), Java (Spring), .NET
- **Base de données** : MySQL, PostgreSQL, MongoDB

Mise en Place / Prérequis

- Serveur web (Apache, Nginx)
- Hébergement centralisé (OVH, AWS EC2, etc.)
- Environnement de développement (IDE, Git, base de données)

2. Architecture Microservices

Avantages

- **Scalabilité** : chaque service peut être déployé séparément
- **Résilience** : la panne d'un service ne bloque pas l'ensemble
- **Flexibilité technologique** : chaque service peut utiliser un langage différent
- **Maintenance facilitée** grâce à l'indépendance des modules

Inconvénients

- **Complexité accrue** (communication entre services, authentification)
- **Surcoût opérationnel** (infrastructure, monitoring)
- **Difficulté de débogage** et de cohérence des données

Technologies Utilisées

- **API Gateway** : Kong, Nginx, AWS API Gateway
- **Communication** : REST, gRPC, GraphQL
- **Orchestration / Conteneurisation** : Docker, Kubernetes
- **Bases de données** : PostgreSQL, Redis, Cassandra
- **Monitoring** : Prometheus, Grafana, ELK Stack

Mise en Place / Prérequis

- Connaissance des **API REST**
 - Maîtrise de **Docker/Kubernetes**
 - Système de déploiement continu (CI/CD)
 - Infrastructure cloud (AWS, GCP, Azure)
-

3. Architecture DApps (Applications Décentralisées)

Avantages

- **Transparence et sécurité** grâce à la blockchain
- **Pas d'autorité centrale**
- **Immuabilité** des transactions (smart contracts)
- **Nouvelle économie numérique** (tokens, DeFi)

Inconvénients

- **Complexité technique élevée**
- **Scalabilité limitée** (selon la blockchain)
- **Difficulté de mise à jour** (smart contracts immuables)
- **Coût des transactions** (frais de gaz)

Technologies Utilisées

- **Blockchain** : Ethereum, Polygon, Solana, BNB Chain
- **Smart Contracts** : Solidity, Rust, Vyper
- **Front-end** : React, Next.js, Vue.js
- **Connexion au réseau** : Web3.js, Ethers.js, MetaMask
- **Stockage décentralisé** : IPFS, Arweave

Mise en Place / Prérequis

- Connaissances de base en **cryptographie et blockchain**
- Environnement Web3 (Hardhat, Truffle, Remix)
- Wallet (MetaMask) pour tester les transactions
- Accès à un réseau de test (Sepolia, Polygon Mumbai, etc.)

Synthèse Comparative

Critère	Client-Serveur	Microservices	DApps
Structure	Centralisée	Modulaire	Décentralisée
Performance	Moyenne	Excellente	Variable
Sécurité	Moyenne	Bonne	Très élevée
Maintenance	Simple	Complexe	Difficile
Scalabilité	Limitée	Très élevée	Moyenne
Technologies clés	PHP, Node.js, MySQL	Docker, Kubernetes, REST	Solidity, Web3.js, Ethereum
Prérequis	Hébergement central	Connaissance API et conteneurs	Connaissances blockchain
Cas d'usage typique	Site vitrine, e-commerce	Plateformes SaaS, grandes applis	DeFi, NFT, DAO

Conclusion

Chaque architecture répond à des **besoins spécifiques** : - **Client-Serveur** : simplicité, idéal pour les petits projets.

- **Microservices** : flexibilité et scalabilité pour les systèmes complexes.

- **DApps** : sécurité et transparence pour les projets basés sur la blockchain.

Le choix dépend donc de la **taille du projet**, de la **complexité fonctionnelle**, et du **niveau de décentralisation souhaité**.